on Windows; for Unix, it's a colon.) If your program is not setuid, you can use the command-line parameter -I to do the same thing.

The Ruby variable $: is an array of places to search for loaded files. As we've seen, this variable is initialized to the list of standard directories, plus any additional ones you specified using RUBYLIB and -I. You can always add directories to this array from within your running program. Prior to Ruby 1.9, this used to be a common idiom:

```
$: << File.dirname(__FILE__)
require 'other_file'
```

This added the directory of the running file to the search path, so other_file.rb could be found there by the subsequent require. Now we use require_relative instead.

```
require_relative 'other_file'
```

Using the Rake Build Tool

Another way to structure code that can be easily invoked from the command line is with a useful utility program called Rake. Written by Jim Weirich, Rake was initially implemented as a Ruby version of Make, the Unix build utility. However, calling Rake a build utility is to miss its true power. Really, Rake is an automation tool—it's a way of putting all those tasks that you perform in a project into one neat and tidy place.

Rake gives you a convenient way to define small tasks and task dependencies, allowing you to say that a particular task requires a different task to run first. Rake also allows you to automate transitions between files based on file extensions, for example, converting all your .csv files to .json.

Let's start with an example. As you edit files, you might accumulate backup files in your working directories. On Unix systems, these files usually have the same name as the original files, but with a tilde character appended. On Windows boxes, the files usually have a .bak extension.

We could write a Ruby program that deletes these files. For a Unix box, it might look something like this:

```
require "fileutils"
files = Dir["*~"]
FileUtils.rm(files, verbose: true)
```

The FileUtils module defines methods for manipulating files and directories. Our code uses its rm method. We use the Dir class to return a list of filenames in the current directory matching the given pattern and pass that list to rm.

Let's package this code as a Rake *task*—a chunk of code that Rake can execute for us.

By default, Rake searches the current directory (and its parents) for a file called Rakefile. This file contains definitions for the tasks that Rake can run.

So, put the following code into a file called Rakefile:

```
desc "Remove files whose names end with a tilde"
task :delete_unix_backups do
  files = Dir["*~"]
  rm(files, verbose: true) unless files.empty?
```